

Engenharia de Software Projeto MALDBANK

Documento: Relatório Técnico do Projeto-Final

Data: 26/06/2026

Alunos: Adriel Alves, Marcus Junior, Davi Israel e Lucas Luiz.

1. Introdução

O MALDBANK é um sistema bancário baseado em console desenvolvido em Java. O sistema visa gerir operações financeiras através de uma arquitetura robusta e orientada a objetos. O sistema possui, gerenciamento de usuários, autenticação de acessos e quatro modalidades de contas: Corrente, Poupança, Salário e Investimento, contando ainda com relatórios de transações e uma ampla cobertura de testes.

2. Processo de Software

2.1 Processo de Software

- **Processo Escolhido:** Processo Incremental com Metodologias Ágeis ([Scrum/Kanban e xp](#)).
- **Justificativa:** Optamos por escolher o modelo incremental, porque ele permite evitar riscos, assim nos permitindo focar na estrutura base do sistema(entidades de utilizadores e contas base) inicialmente. Após implementarmos a fundação do sistema, poderíamos implementar as regras de negócio de maneira mais segura, como o cálculo de investimentos e os serviços polimórficos de pagamento.

2.2 Papéis e Responsabilidades

- **Adriel: PO e Scrum Master**, Engenharia de Requisitos, Mapeamento do Escopo, Documentação Técnica, gerenciamento do quadro kanban, distribuição de tarefas, elaboração de diagramas e histórias de usuário.
- **Marcus: Scrum Master Substituto e DEV**, Implementação do Core (Entidades), implementação dos métodos de transação e validação de QA de algumas tarefas.
- **Davi: DEV**, Desenvolvimento da Camada de Serviços (Services), Validação de QA de algumas tarefas, Regras de Persistência (Repositories) e Implementação da interface via terminal.
- **Lucas: DEV**, Desenvolvimento da Camada de Serviços (Services), Regras de Persistência (Repositories) e Qualidade (Desenvolvimento de Testes).

3. Requisitos de Software

3.1 Catálogo de Requisitos

- **RF001 (Funcional):** O sistema deve permitir o cadastro de um usuário contendo Nome, CPF, Senha, Tipo de conta.
- **RF002 (Funcional):** O sistema deve associar cada usuário a uma conta bancária tipada através de uma enumeração (TipoConta).
- **RF003 (Funcional):** O sistema deve permitir realizar saques e depósitos, verificando o saldo corrente e atualizando o histórico.
- **RF004 (Funcional):** Os diferentes tipos de Conta devem possuir suas características únicas implementadas.
- **RF005 (Funcional):** A classe Investimento deve calcular o valor de retorno com base no tempo (dias) e taxas específicas.
- **RF006 (Funcional):** A Conta Poupança deve disponibilizar um método para render juros baseados.
- **RNF001 (Não Funcional):** Regra de Negócio/Segurança: O saldo de nenhuma conta poderá ficar negativo durante as operações.
- **Portabilidade:** O sistema não deve exigir instalação de banco de dados ou servidores locais, funcionando de forma independente através da linha de comando.

3.2 Especificação de Requisitos (História de usuário)

História de Utilizador: US04 — Operações de Investimento Contratado

Funcionalidade	História
-Ver Saldo	Como usuário do banco, eu Gostaria de poder verificar quanto dinheiro tenho em minhas contas, de forma fácil, Para que eu possa acompanhar minhas economias.
-Efetuar Transferências	Como usuário do banco, eu Gostaria de poder transferir meu dinheiro rapidamente de forma pratica e facil, Para que eu não precise sacar e ir pessoalmente pagar.
-Sacar Saldo	Como usuário do banco, eu Gostaria de visualizar meu saque Para que eu receba meu dinheiro.
-Ver Extrato	Como usuário do banco, eu Gostaria de visualizar o histórico de movimentação das minhas contas Para que eu consiga consultar para onde vai o meu saldo.
-Efetuar Depósito	Como usuário do banco, eu

	Gostaria de poder depositar meu dinheiro de forma rápida Para que meu saldo melhore.
-Conta Investimento	Como usuário do banco, eu Gostaria de aplicar uma fração do meu saldo, em opções de renda passiva (como CDB ou FII), Para que eu possa render dinheiro e projetar um bom retorno financeiro.

Critérios de Aceitação: EX

1. O valor destinado ao investimento não pode, sob qualquer circunstância, ser superior ao saldo disponível em conta.
2. A data estipulada para retirada deve ser obrigatoriamente uma data futura em relação ao momento da aplicação.
3. Caso o ativo informado não seja reconhecido pelo sistema, a operação deve ser abortada exibindo uma mensagem de erro.

4. Modelo de Software

O modelo estrutural detalhado do sistema foi construído baseado nas especificações da UML. O diagrama de classes, fluxograma de interação e o diagrama de casos de uso a seguir demonstra a herança polimórfica das contas estendendo a classe abstrata, a interface operacional de serviços Pagamento, e as relações estruturais com as entidades.

PS: Como são relativamente grandes, optei por colocar todos no README.md

[Diagramas](#)

5. Princípios e Propriedades de Projeto

A conformidade arquitetural do projeto MALDBANK foi atingida através da aplicação dos fundamentos de design orientado a objetos e princípios SOLID:

- **Integridade Conceitual:** Mantida pela centralização do ponto de inicialização na classe App, que coordena o fluxo de menus ao controlador do terminal Main.
- **Coesão:** Todo o desenvolvimento foi implementado seguindo padrões e regras impostas no repositório, para que parece ter sido desenvolvido unicamente por uma pessoa.
- **Baixo Acoplamento:** Introduzido no sistema por meio da interface de serviço Pagamento. Desta forma, componentes do core negocial acoplam-se a abstrações

estáveis (contratos de métodos) em vez de acoplarem-se a classes concretas mutáveis.

- **Ocultação de Informação (Encapsulamento):** Dados sensíveis e estruturais como senhas, CPFs e saldos utilizam tipagem restritiva(`private` e `protected`), impedindo acessos externos diretos. A leitura e escrita são feitas exclusivamente por métodos controlados.
- **Princípios SOLID Aplicados:**
 1. **S (Responsabilidade Única):** Divisão entre as entidades/classes/services e suas responsabilidades, evitando múltiplas responsabilidades nas mesmas, EX, (User, Usuario) e os mecanismos responsáveis pela gestão de sua persistência técnica (UserRepository).
 2. **O (Aberto/Fechado):** A superclasse abstrata Conta está fechada para modificações em sua estrutura, porém está aberta para extensões polimórficas (como a criação de novos tipos de contas).
 3. **L (Substituição de Liskov):** Subclasses especializadas (ContaSalario, ContaCorrente) herdam e respeitam comportamentos da classe base Conta, podendo substituí-la sem induzir erros no sistema.
- **Preferência por Composição em vez de Herança:** Embora a herança seja empregue para especializar o comportamento operacional das contas, a classe Conta utiliza a composição forte para acoplar a entidade Usuario e agregar a coleção dinâmica de históricos por meio de uma lista estruturada de instâncias de Transacao (List).
- **Lei de Demeter:** Os métodos implementados evitam o conhecimento e navegação em estruturas internas profundas de objetos terceiros (cadeias de chamadas). No método investir da classe ContaInvestimento, o bloco lógico interage exclusivamente com o parâmetro de entrada direto fornecido, invocando apenas investimento.getValor().

6. Testes de Software

6.1 Testes Unitários

A verificação da corretude das regras de negócio foi garantida através do desenvolvimento de baterias de testes automatizados com o framework JUnit. O foco concentrou-se no isolamento lógico e validação de caminhos felizes e de exceção, englobando:

1. Garantia de interrupção e lançamento de alertas operacionais ao tentar efetuar saques cujo montante ultrapasse o saldo líquido atual da conta.
2. Exatidão matemática de ponto flutuante na projeção temporal de juros diários simulados dentro do método calcularRetorno() para ativos CDB, LCI e Tesouro Direto.

6.2 Uso de Mocks para Simulação de Dependências

Para assegurar o isolamento estrito dos testes aplicados sobre a camada de serviços em UsuarioService, utilizou-se o framework Mockito para a criação de componentes simulados (Mocks) correspondentes à classe UserRepository. Esta abordagem permitiu inspecionar e testar a integridade das funções de cadastro, deleção e atualização de utilizadores, validando o fluxo de dados sem a necessidade de instanciar ou depender do comportamento real de armazenamento em coleções de memória ou infraestruturas externas de repositório durante a fase de testes unitários.

Links: [Trello](#) , [GitHub](#) , [Diagramas](#) , [Regras](#).